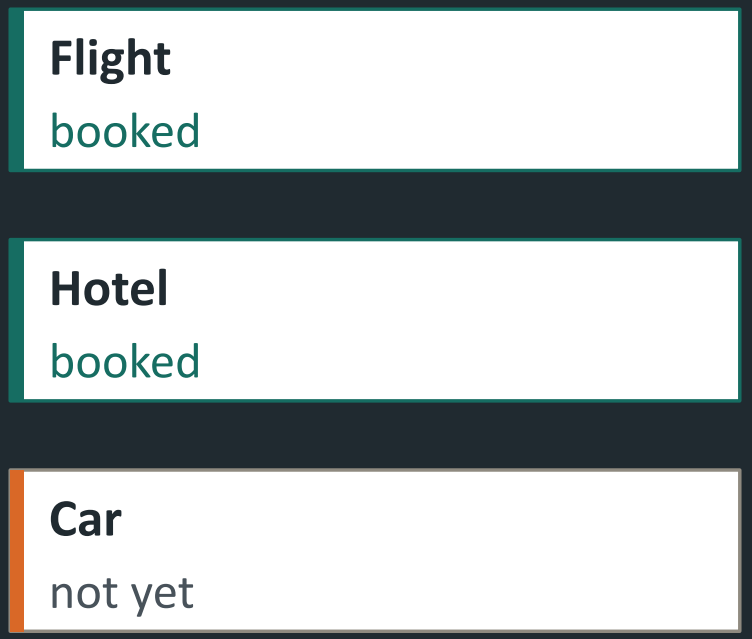
Code that survives the crash

Durable Task Scheduler
for workflows and AI agents



Tamir Dresher · Update Conference Prague 2026

Flight booked. Hotel booked. No car.

1

1

0

Flight
booked

Hotel
booked

Car
not booked

The process died before the car booking.



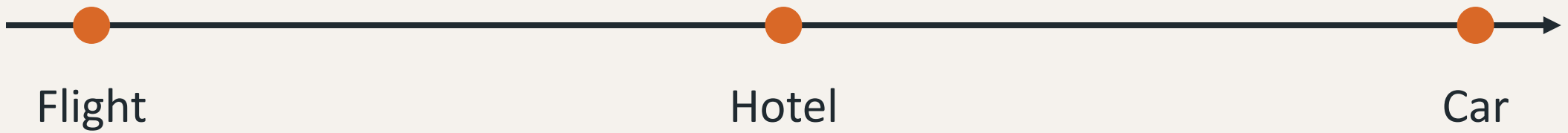
What got booked? Will a rerun book twice?

Three awaits. Who remembers the progress?

NaiveTrip.BookAsync(booking, trip, attemptId) · new attemptId per run

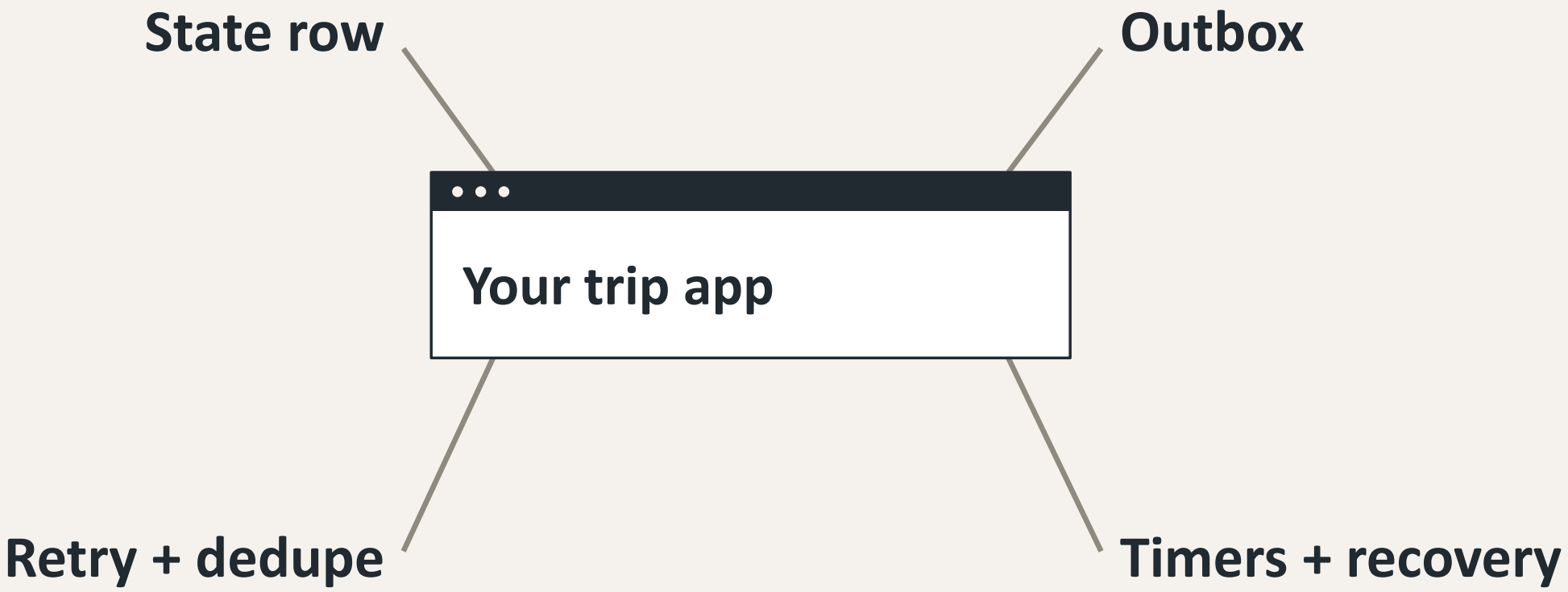
LIVE DEMO

```
BookingReceipt flight = await booking.BookFlightAsync(
    new($"{attemptId}/flight", "flight", trip));
BookingReceipt hotel = await booking.BookHotelAsync(
    new($"{attemptId}/hotel", "hotel", trip));
BookingReceipt car = await booking.BookCarAsync(
    new($"{attemptId}/car", "car", trip));
```



Rerun: 2 flights · 2 hotels · 1 car

Your trip app just hired a workflow team.



Congratulations. You also own a workflow engine.

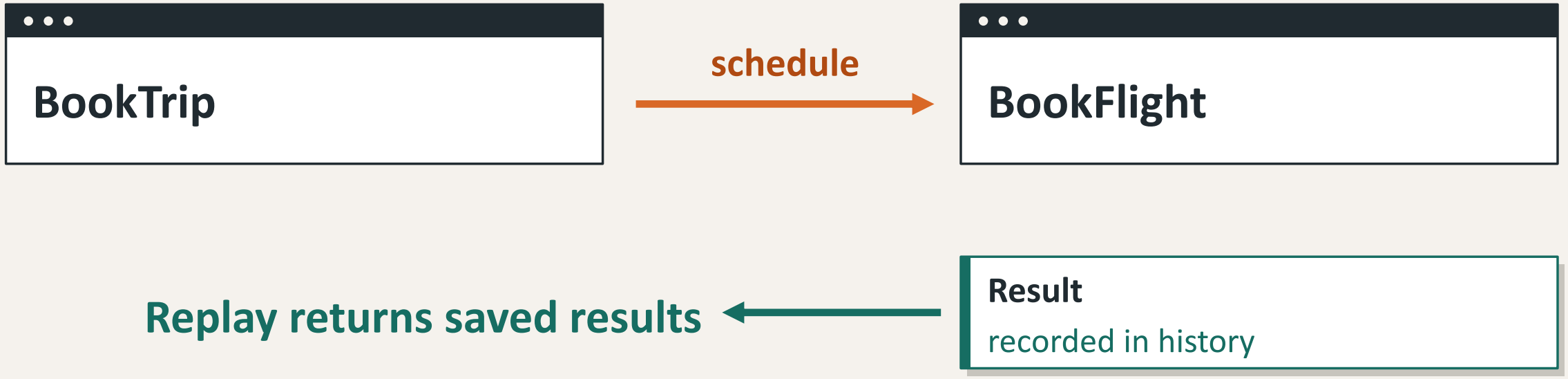
Keep the code. Give the trip a durable memory.

TaskOrchestrator

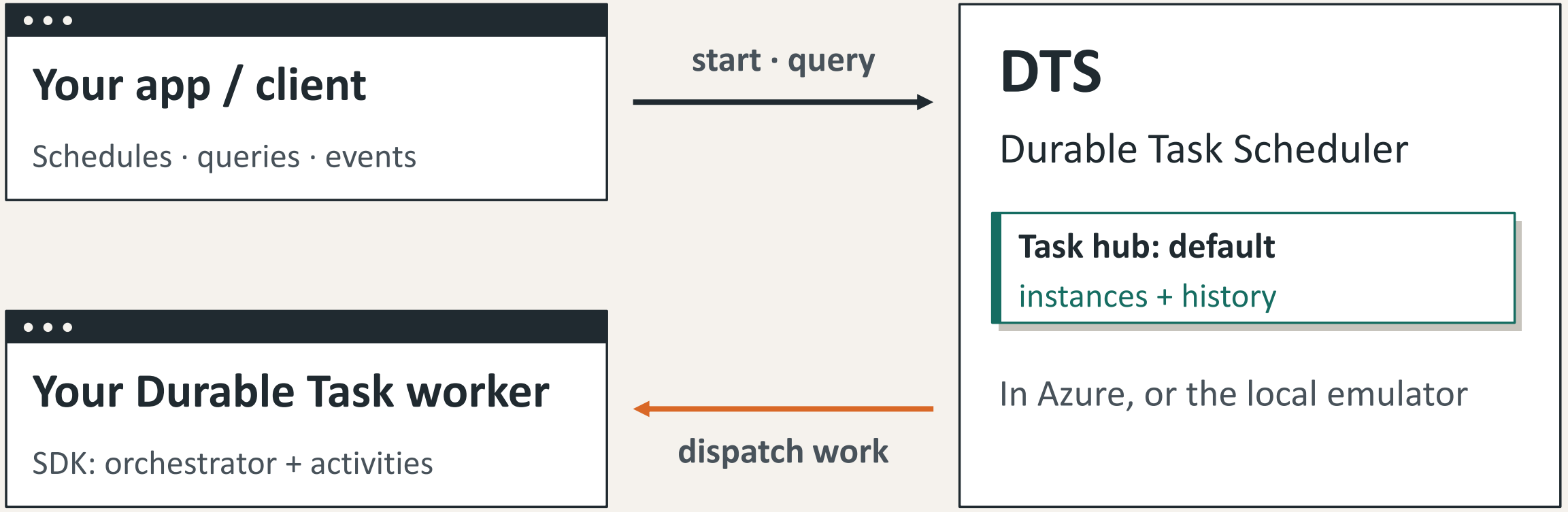
Decide the next step

TaskActivity

Execute I/O



Your worker runs C#. DTS keeps the history.



DTS does not host your C# methods.

The flight await creates a recorded boundary.

`BookTrip.RunAsync(context, trip)` **WORKER RUNNING**

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```

DURABLE HISTORY

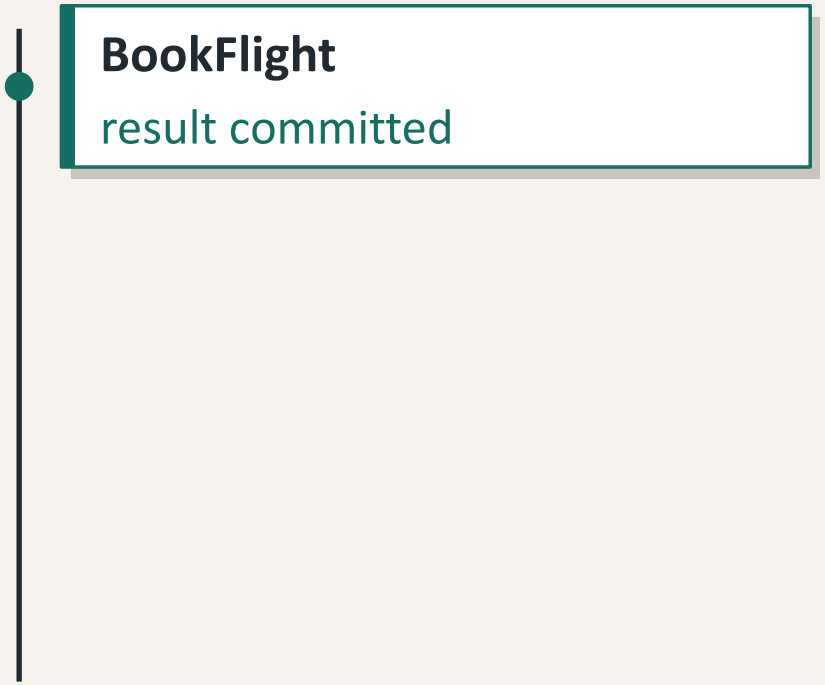
BookFlight
scheduled

New command → dispatch the activity.

The flight receipt lands in history.

BookTrip.RunAsync(context, trip) WORKER RUNNING

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```



Completion + result are committed.

Hotel recorded. Car still unbooked.

`BookTrip.RunAsync(context, trip)` **WORKER RUNNING**

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```

DURABLE HISTORY

- **BookFlight** — result committed
- **BookHotel** — result committed
- **BookCar** — scheduled / pending

Flight + hotel recorded. Car still 0.

Kill the worker. Keep the two receipts.

`BookTrip.RunAsync(context, trip)` **WORKER DOWN**

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```

DURABLE HISTORY

- **BookFlight** — result committed
- **BookHotel** — result committed
- **BookCar** — scheduled / pending

Stack lost. Two committed results retained.

Restart at line one. Return the saved results.

`BookTrip.RunAsync(context, trip)` **NEW WORKER**

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```

Flight + hotel came back from history

DURABLE HISTORY

BookFlight
result committed

BookHotel
result committed

BookCar
scheduled / pending

Code runs again. Completed bookings aren't reissued.

Continue the car. Finish 1 / 1 / 1.

`BookTrip.RunAsync(context, trip)` **NEW WORKER**

```
BookingReceipt flight = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookFlight), trip);
BookingReceipt hotel = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookHotel), trip);
BookingReceipt car = await
  context.CallActivityAsync<BookingReceipt>(
    nameof(BookCar), trip);
```

The car runs for real: 1 / 1 / 1

DURABLE HISTORY

BookFlight
result committed

BookHotel
result committed

BookCar
result committed

Same instance. Three bookings. One of each.

Pull the plug between hotel and car.

LIVE DEMO

Resume the trip. Not a new trip.

1

Same instance ID

2

Two saved results

3

1/1/0 → 1/1/1

Watch those three things, then the dashboard timeline.

Will the same trip choose the same next call?

Inside BookTrip.RunAsync(context, trip):

```
if (DateTime.Now.Hour < 17)
    await context.CallActivityAsync<BookingReceipt>(
        nameof(BookFlight), trip);
else
    await context.CallActivityAsync<BookingReceipt>(
        nameof(BookHotel), trip);
```

FIRST RUN · 16:59

History: BookFlight

REPLAY · 17:01

Code: BookHotel

Same code. Different branch. History and command no longer match.

Replay needs the same decisions.

Inside the orchestrator:

Replay-safe: use the context

```
DateTime created = context.CurrentUtcDateTime;
Guid bookingId = context.NewGuid();
```

Not replay-safe: move it into an activity

DateTime.Now · Guid.NewGuid() · Random · HTTP · database / file I/O

Replay reconstructs decisions—not arbitrary side effects.

Hotel: two attempts. One reservation.

- **Hotel commits**
  provider owns receipt
- **ACK is lost**
  worker crashes
- **Hotel repeats**
  same operation key

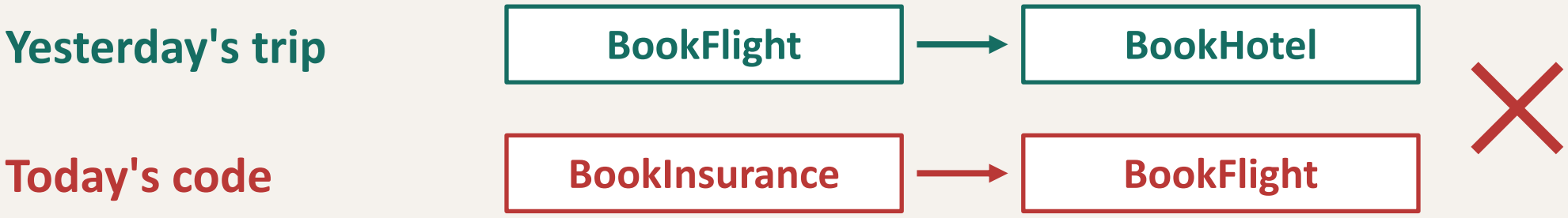


Inside BookTrip.RunAsync(context, trip), the hotel call passes:

```
new BookingCommand(
    $"{context.InstanceId}/hotel", "hotel", trip)
```

“Exactly once” = saved result + stable key + provider dedupe.

Add insurance today. Don't break yesterday's trips.



Client registration: new trips start as version 2.0

```
client.UseDefaultVersion("2.0");
```

Inside BookTrip.RunAsync(context, trip): pre-upgrade trips skip the new step

```
if (context.CompareVersionTo("2.0") >= 0)
    await context.CallActivityAsync<BookingReceipt>(
        nameof(BookInsurance), trip);
```

Old trips replay the old path. New trips get insurance.

Retry the flight call, not the entire trip.

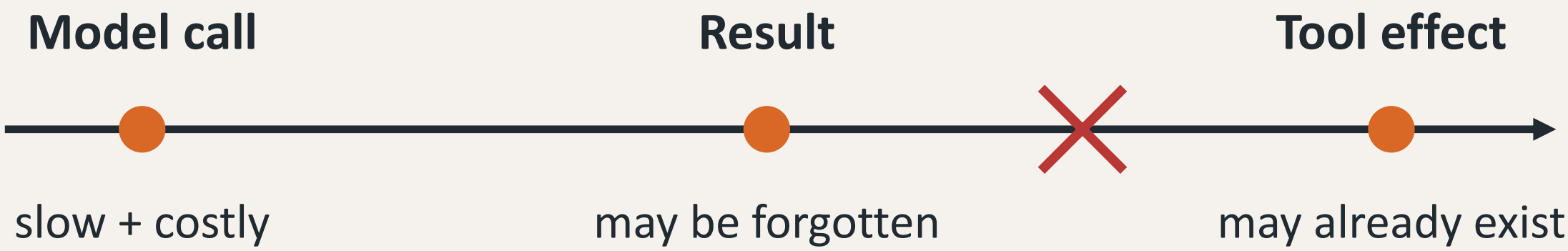
Inside BookTrip.RunAsync(context, trip):

```
BookingReceipt flight = await
    context.CallActivityAsync<BookingReceipt>(
        nameof(BookFlight),
        new BookingCommand(
            $"{context.InstanceId}/flight", "flight", trip),
        TaskOptions.FromRetryPolicy(
            new RetryPolicy(3, TimeSpan.FromSeconds(1))));
```



Retry the work. Don't multiply the business effect.

Agents make forgotten work more expensive.



Conversation memory is not workflow recovery.

Swap the provider, not the workflow.



Inside ConfigureDurableAgents(options => …); chatClient is any IChatClient:

```
var planner = chatClient.AsAIAgent(
    instructions: "Plan a trip within the budget.",
    name: "TripPlanner");
options.AddAIAgent(planner);
```

Demo model is scripted. Swap the IChatClient; the workflow stays.

Planner output becomes the reviewer's input.

Inside PlanAndBookTrip.RunAsync(context, input):

```
DurableAIAgent planner = context.GetAgent("TripPlanner");
AgentSession plannerSession =
    await planner.CreateSessionAsync();
AgentResponse itinerary = await planner.RunAsync(
    JsonSerializer.Serialize(input.Trip), plannerSession);
DurableAIAgent reviewer = context.GetAgent("BudgetReviewer");
AgentSession reviewerSession =
    await reviewer.CreateSessionAsync();
AgentResponse review = await reviewer.RunAsync(
    itinerary.Text, reviewerSession);
```

TripPlanner → BudgetReviewer → immutable itinerary

Hold a deadline, not a worker thread.

Inside PlanAndBookTrip.RunAsync(context, input):

```
DateTime deadline = context.CurrentUtcDateTime.AddMinutes(15);
using CancellationTokenSource timerCancellation = new();
Task timeout = context.CreateTimer(deadline, timerCancellation.Token);
Task<ApprovalSignal> approval =
    context.WaitForExternalEvent<ApprovalSignal>("TripApproval");
if (await Task.WhenAny(approval, timeout) == timeout)
    return "TimedOut";
timerCancellation.Cancel();
```



hold / deadline



approval event

Wake-up ≠ authorization. Cancel the losing wait.

Crash the agents. Don't pay for the planner twice.

LIVE DEMO · Kill the worker while it waits for the traveler



Saved results come back from history. A call that was mid-flight runs again.

A model can propose. A traveler must authorize.



Frozen plan + deadline + trusted validation

A second model's agreement is not authorization.

Choose the boundary you want to own.

	BEST WHEN	YOU STILL OWN
Hangfire	Background jobs and continuations	Job code, storage, workers and retry-safe effects
MassTransit saga	Correlated messages and explicit process state	State machine, persistence and message contracts
Temporal	Code-first workflows on Temporal's runtime	Workflow/activity code, workers, replay compatibility
Durable Functions	Workflows hosted in Azure Functions	Function code, hosting plan and backend configuration
Durable Task + DTS	Code-first workflows on your worker compute	Worker host, workflow code, hub access (RBAC), safe effects

Sometimes a queue is exactly enough.

One short idempotent job	✓ **Queue / Hangfire**
One atomic data change	✓ **Database transaction**
An explicit message-driven process	✓ **Keep the saga**

No durable coordination requirement? Don't add one.

Same trip. Same crash. In Python.

From python/book_trip.py (durabletask-python, same emulator):

```python
def book_trip(ctx: task.OrchestrationContext, trip: dict):
    flight = yield ctx.call_activity(
        book_flight, input=f"{ctx.instance_id}/flight")
    hotel = yield ctx.call_activity(
        book_hotel, input=f"{ctx.instance_id}/hotel")
    car = yield ctx.call_activity(
        book_car, input=f"{ctx.instance_id}/car")
    return {"trip": trip, "bookings": [flight, hotel, car]}
```

Killed before the car, new worker: flight 1 · hotel 1 · car 1

Same crash, same resume. Python yields where C# awaits.

Test the failure boundaries before the happy path.

1 **Crash at the boundary**

Can you explain every receipt?

2 **Repeat one effect**

Can the owner deduplicate it?

3 **Upgrade a waiting instance**

Does yesterday's run still finish?

Start with one workflow you cannot afford to forget.

- What must survive?
- What may repeat?
- Who owns the wait?



Slides + code

github.com/tamirdresher/DurableTaskUpdatePrague2026

Q&A